

Agent-0 Implementation Roadmap and Staffing Plan

实现

1. Harness框架选择

Harnesses定义

An agent harness is the software infrastructure surrounding an AI model that manages everything except the model's actual reasoning. It acts as the intermediary between the LLM and the outside world, handling tool execution, memory storage, state persistence, and error recovery. By maintaining context across multiple sessions, a harness transforms a simple text generator into a capable, long-running agent.

简而言之：An agent harness is everything that wraps around an LLM (tool execution, memory, context management, state persistence), excluding the model itself.

关于Harness 和 LLM在agent中的关系，一个比较典型的类比：

What is an Agent Harness?

An agent harness wraps around a model to manage long-running tasks reliably. The model generates responses; the harness handles everything else.

The Model is the Engine



Generates raw power and responses. Probabilistic and powerful, but lacks control.

The Harness is the Car



Provides steering, brakes, and structure. Manages approvals, filesystem access, sub-agents, and errors to reach a destination reliably.

过去workflow agent模式正在被harness engineering + LLM的模式所取代，其中80%由模型承担，20%由harness承担：

handled by model (80%)

handled by harness (20%)

- define agents
- route messages
- manage task lifecycle
- handle dependencies
- spawn/terminate

- persistence
- replay
- cost control
- observability
- error recovery

基于上述定义，我们在Harness Engineering开发工作中，如果遇到智能体犯错，应该首先考虑对环境进行工程改造，使其在物理层面上不会再次犯同样的错误——将每个agent错误都视为可以永久修复的工程问题，而不是重试prompt^[5]。

这种问题解决模式意味着2件事儿：

1. 更新agent指令文件（AGENTS.md 或者 类似文件），添加可以防止已知故障模式再发生的规则。每一项对应一个已知故障，然后持续维护该指令文件。
2. 构建可以机制化验证行为正确性的工具。

i.e. 将每次智能体的失败都视为需要永久预防的系统性问题，每种失败模式的出现都会促使指令文件的更新或催生一个新工具（例如，如果agent在测试UI交互时反复失败，就构建一个截屏工具协助失败处理），以机制化的手段来确保智能体任务执行的正确性。从运行环境、可访问的范围、需要验证的内容，以及出错时强制重试的机制来约束智能体行为（guardrails）。

Harnesses组成

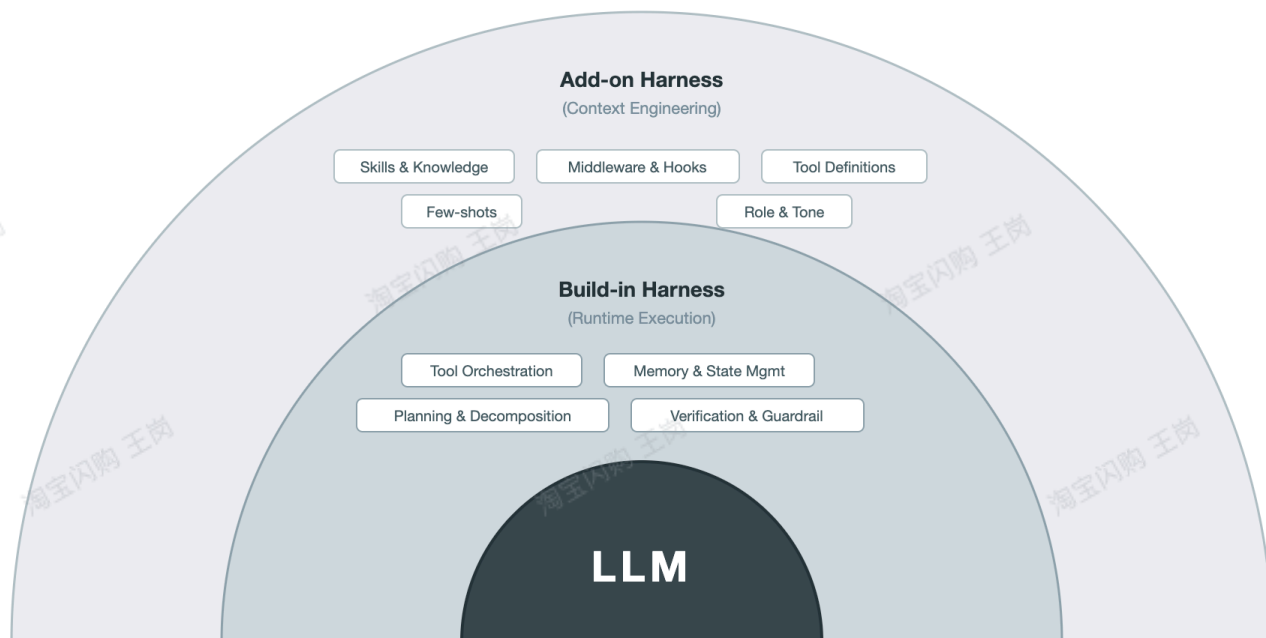
首先我们看一下，相较于过去prompt engineering工作，harness engineering需要做些什么：

类别	Prompt Engineering	Context Engineering	Agent Harness
定义	一次模型请求的输入优化	每一轮的模型上下文窗口应该保留/过滤的信息	除了Prompt和Context Engineering工作外，还处理上下文窗口之外的部分内容，包括：工具执行、状态存储、会话管理和错误修复。

我们可以将Harness功能模块大致拆成下述6块（参考claude code）：

序号	优先级	功能	
1	★★★★★	context engineering	

2	★★★★★	tool orchestration	
3	★★★★★	memory & state management	working context (t
	★★★★		session state (a d
	★★		long-term memory
4	★★★★★	planning & decomposition	
5	★★★★★	verification & guardrails	
6	★★	lifecycle management	



Harness Engineering — Concentric Architecture

Layer 1: LLM

Layer 2: build-in Harness (runtime execution): tool orchestration, memory&state management, planing&decomposition, verification&guardrail

Layer 3: add-on Harness (context engineering): skills&knowledge, few-shots, middleware&hooks, tool def, role&tone

📎 harness_arch.html

我们以claude code作为标杆（harness block结构详情可以参考：

📄 [Claude Code Harness 设计](#)），引入了如下备选的开源框架（因为需要对系统prompt做改造，所以闭源框架并不适合）：[📄 Harness框架对比](#)

2. Harness结构选择

Harness典型结构

1. Single-agent supervisor

一个模型在一轮循环中配备工具、记忆和验证功能。

该框架负责了初始化、上下文注入、工具分发、状态持久化和清理的各环节管理。

这种结构非常适合处理边界清晰的任务（bounded tasks），例如一个具备知识库和工单系统的客服助手。

2. Initializer-exector split

初始化器（initializer）执行一次，并且建立持久化的项目环境（project environment）：文件夹结构、功能列表、init.sh，以及初始化git commit。每一个执行器都会从该环境（project environment）读取数据并完成一项独立功能的逐步开发、运行测试、提交代码、更新进度文件，然后清空退出。项目环境（project environment）是所有会话共享的记忆存储。

Claude Code采用的是这种两步解决方案：

1. **Initialize agent**: 第一个agent会话使用特殊prompt来让模型设置初始环境：一个 `init.sh` 脚本、一个 `claude-progress.txt`（用于记录agents已经完成的操作），以及一个初始 `git commit`（用于显示已添加文件）。
 - a. Feature list: 为了解决智能体one-shotting an app 或者过早认为一个项目已经完成的问题，我们鼓励initialize agent编写一份全面的功能需求文件，以扩展用户的原始指令。
 - b. Incremental progress: 在提供初始环境脚手架后，在下一轮的coding agent每次只会被分配一项功能开发。这种“渐进”方式被证明对于解决agent倾向于一次性做太多事情的问题，非常有效。实验显示，要求模型使用描述性的提交信息将开发进度commit to git，并在进度文件中写入进度摘要，是agent实现操作回滚、错误修复的最佳方式。
 - c. Testing: agent倾向于在没有进行充分测试的情况下将某个功能开发标记为已完成（最主要的错误模式之一），通过提供特定测试工具，可以显著提高智能体的任务完整性表现——这样agent可以识别并修复哪些仅从代码语法层面无法明显看出的错误。

```
[Assistant]. I'll start by getting my bearings and understanding the current state of the project.  
[Tool Use]. <bash - pwd>
```

```
[Tool Use] <read - claude-progress.txt>
[Tool Use] <read - feature_list.json>
[Assistant] Let me check the git log to see recent work.
[Tool Use] <bash - git log --oneline -20>
[Assistant] Now let me check if there's an init.sh script to
restart the servers.
<Starts the development server>
[Assistant] Excellent! Now let me navigate to the application
and verify that some fundamental features are still working.
<Tests basic functionality>
[Assistant] Based on my verification testing, I can see that
the fundamental functionality is working well. The core chat
features, theme switching, conversation loading, and error
handling are all functioning correctly. Now let me review the
tests.json file more comprehensively to understand what needs
to be implemented next.
<Starts work on a new feature>
```

2. **Coding agent**: 后续的每一个会话会要求模型逐步取得进展，并实现记忆/状态（memory/state）文件的结构化更新（`claude-progress.txt`、`git commit`）。快速进入状态的操作：

- 执行`pwd`查看当前工作目录，了解可以获取的文件系统现状；
- 读取`git logs`和`progress files`，了解最新的工作进展
- 阅读`feature list file`并挑选尚未完成的功能中优先级最高的项目推进。

3. Multi-agent coordination

对于复杂项目，该结构harness会指派特定智能体（例如researcher、writer、reviewer），管理智能体之间的工作交接，这样每一个智能体可以从过去执行步骤中获取到相关上下文，排除无关历史信息的影响。

3. Harnesses开发方法

固定框架、结构，模型锁定后，选择业务场景之后，就到了开发环节。

Principle 1: Model和Harness的一体化开发，Harness先行，不约束在EBM和Qwen上

为什么首先是harness？因为harness相较于llm，存在下述4点不可替代的作用。同时，由于模型agentic capability本身具备的通用性，LLM越来越被作为跑pluggable commodity：

原因	说明
Capability Expansion	模型越强能做的事儿越多 → 能做的事儿越多带来的错误模式也更多样化 → 更多样化的错误模式需要更复杂的错误处理机制 → 错误处理机制由harnesses负责。
Cost Optimization	更强的模型成本也更高。好的harnesses设计会把简单任务路由到低成本模型，复杂任务路由给高成本模型，实现边际效用最大化。 这些都应该通过测试校准，而不是无脑的取SOTA。
Reliability requirements	生产环境需要达到99.9%的可用性（任务能正常运行的时间/任务数占比）。模型是基于概率预测的，Harnesses用过重试逻辑、回退和验证机制保证了任务的可靠性。
Organizational integration	模型无法处理身份验证、权限控制、速率限制、合规检查。Harnesses可以。

以Manus为例，在6个月内重写了5次，但模型基座并没有变化。每一次架构的重写都显著提升了manus的可靠性和任务完成率；

而Vercel最初配备了功能齐全的工具库，涵盖搜索、代码、文件和API工具的方方面面。但结果却令人失望。Agents在执行过程中变得困惑、工具调用冗余、无必要的执行步骤泛滥。然后，他们去除了80%的agent工具并且获得了更好的结果。

模型之间有能力差异，主要通用一些agentic capabilitiy benchmark体现，但我們需要注意2点：

1. 这种差异化优势具备显著的迁移性，在上述benchmark表现好的模型，无论采用何种框架，都表现出更好的效果（例如claude、Kimi-k2.5）。
2. 这种差异会被customized harness的适配性放大（例如qwen和claude之间；但可能是因为kimi消费了大量claude code合成数据，kimi系列在claude code上和claude相比，表现差距并没有显著放大）

Principle 2: 从简单的原型开始，从场景下的单一任务开始，“由俭入奢”

框架和结构都是用既有的方案，所以我们并不需要花时间在原型初始化上。我们一开始就有可在开发环境执行的agent实例，一旦场景确定，基于场景的模拟环境构建完毕，主要工作就变成了找任务、跑数据、发现失败模式、找harness解决办法，如此循环，直到我们既定的测试目标全部成功。

序号	步骤	说明
1	Start with one task end-to-end	选择一个具备交付价值的agent任务，基于该任务构建最小化的支撑框架以确保其可靠性（MVP），然后部署上线，直接从生产环境反馈中学习迭代。 概括起来：价值导向 → 轻量起步 → 快速上线 → 数据驱动
2	Instrument everything	对所有环节做监控，记录每一次工具调用、错误环节、人为干预、超时情景。 只有可以被准确衡量的点，才有改进可能。
3	Iterate based on failure modes	每一次故障都暴露出防护措施缺失的点。添加防护措施，部署，修复，然后寻找下一个故障，如此循环。
4	Measure outcomes, not activity	衡量结果，而非活动量——最总任务完成情况，而非token数。衡量业务满意度，而不是速度。优化任务可靠性，而不是任务功能丰富度。

Principle 3: 不要想着如何运行成功，而是想着什么是成功的情景，通过情景测试找到当前agent和期望的成功之间的差异，指导更新

claude code作为比较通用的agent代表，Boris及其团队在开发中总结了如下4个比较常见的失败场景，以及他们对应的解决办法，对我们也比较有启发：

问题描述	Initializer Agent解决办法	Coing Agent解决办法
agent过早认为整个项目目标已经达成	设置feature list file：根据输入规范，设置一个结构化的JSON文件，其中包括了end-to-end功能描述。	在会话开始时读取feature list file，并选择一个功能进行开发。

agent在上轮执行后遗留的环境存在bugs或者未记录的进展	初始化git repo、将过程记入progress notes file	会话开始时读取progress notes file和git commit日志，并且在开发服务器上运行基本测试，以发现任何未记录的bugs。 会话结束时，提交最新的git commit并更新progress note files
agent过早的认为功能开发已经实现	创建a feature list file	self-verify所有功能。只有在仔细测试后，才能将功能标记为“passing”
agent需要花费时间掌握如何运行app	编写一个 <code>init.sh</code> 脚本，用于启动开发服务器（自动完成依赖安装、环境变量设置、代码编译等）。	会话开始时先读取 <code>init.sh</code>

4. 场景实现

业务场景的选择虽然是开放的，但必须遵循2个基本条件^[6]：

1. 可直接、准确衡量任务成功与否
2. 可见的生产力价值导向^[6]

注：所以，任何“xxx分析”、“xxx诊断”、“xxx报告”类的任务，场景上都应该抵制；可验证的场景做好，这些场景自然会有迁移提升。

以营销活动报名和活动配置场景为例：

序号	分块9	草稿	优先级
----	-----	----	-----

1	业务流程摸索及实现轨迹设计	- 活动感知：遍历活动列表，获取活动和店铺信息匹配情况 - 资格决策：检查商家资质、历史违规记录、活动互斥规则（例如“满减”和“折扣”活动不能同时报名） - 最优化配置：在成本控制前提下最大化曝光 - 配置验证：确认活动生效时间、优惠力度、库存限制，同步商家端展示	
2	环境仿真实现	- 活动列表：返回活动类型、报名条件、预估流量登记、覆盖城市 - 冲突检测器：模拟活动叠加时的风险（如优惠力度过高导致亏损的预测） - 流量模拟器：根据报名活动类型反馈预估曝光	
3	边际场景盘点、测试样本合成	- 资格瞬变：报名过程中活动名额已满活资质要求变更（如评分骤降） - 时间冲突：活动有效期重叠但优惠互斥，需自动调整排期 - 成本溢出：多活动叠加后优惠力度超过商家预设预算阈值 - 新手期过度：模拟90天扶持期结束前后的策略切换（流量活动 → 常规活动）	
4	CPT+ SFT + RL with rejection sampling	- cpt对harness prompt可以内化的知识进行内化，降低context window占用 - sft对harness框架及流转逻辑进行内化 - reward设计：采用多目标加权（活动成功率 × 0.4 + 曝光量提升 × 0.3 + 成本控制 × 0.2 + 操作步骤数惩罚 × 0.1） - rj策略：过滤导致商家亏损活违规的轨迹，保留“低沉本高曝光”的pareto最优解 - 冷启动约束：学习“先查资格再报名”的硬规则、人工介入阈值触发时机等 - 多模态需求？	

待办

1. 论题1: Harness框架选择讨论

负责人：芒鞋，羊灵，晨跃，诚斋

name	language	git repo	assessment	score
Kimi Code (即 Kimi Cli)	python + TypeScript	https://github.com/MoonshotAI/kimi-cli		
OpenHands (即 OpenDevin)	python + TypeScript	https://github.com/All-Hands-AI/OpenHands	1. 原生支持 CodeAct架构, 与Claude Code 一致 2. 完整记录了 observation → thought → action → codeexecution 全链路轨迹 3. 支持docker环境 隔离, 并行批量 执行 4. 支持任意基座模型	
openclaw	TypeScript + Swift	https://github.com/openclaw/openclaw		
SWE-agent	python	https://github.com/SWE-agent/SWE-agent	1. 针对github代码 仓库编辑优化	
OpenAI Codex	Rust	https://github.com/openai/codex		
Goose	Rust + TypeScript	https://github.com/block/goose		
deerflow-2.0	Python + TypeScript	https://github.com/bytedance/deer-flow		

2. 论题2: Harness结构选择讨论

结论： `initializer-executor split`

整个agent工作过程大致分为两个阶段：

- 阶段1：项目启动时运行一次。它会初始化环境、创建结构化的任务列表、初始化版本控制，并将这些内容写入一个进度文件，供后续会话读取。
- 阶段2：之后重复执行。每个会话都会加载状态信息、选择下一个未完成的任务并逐步完成它，然后再退出当前任务前保存进度。任何会话都不需要知道之前会话中发生了什么，因为harness会将这些信息外化到文件和提交的commit历史中。

注：模型从不直接访问外部系统——在每个会话结尾，harness将已经完成的操作和后续待执行操作存储为结构化的记录。即使每次模型调用都是从头开始，项目也会跨会话累计记忆信息。

3. 论题3: harness开发依赖的基模选择

负责人：羊灵，芒鞋，晨跃，诚斋

模型	尺寸	是否开源	弹内部署	模型agent能力 (0-5分)	可训练版本	训练思路	
Qwen3.5	397B-A17B	✓	✓	2	35B-A3B 27B	直接训练	
Kimi-k2.5	1TB-A32B	✓	⊘	4	无	基于qwen基座，对Kimi-k2.5做agentic数据蒸馏，提升基础能力后再训练agent-0 model	
GLM-5	744B-A40B	✓	⊘	4	无	同上	
Mini Max-M2.7	-	⊘	⊘	4	无	同上	

Clau de	-			5	无	基于qwen基座 claude蒸馏版本训 练： Jackrong/Qwen3.5-35B-A3B-Claude-4.6-Opus-Reasoning-Distilled Jackrong/Qwen3.5-27B-Claude-4.6-Opus-Reasoning-Distilled	
...							

4. 论题4: 业务场景选择

负责人：芒鞋，羊灵，晨跃，诚斋

类别	场景	具体操作
店铺运营管理	菜品上下架	节假日/售罄时快速切换状态
	起送价/配送费修改	根据时段/天气动态调整
订单处理类	异常订单识别	地址不详、超远距离、疑似
用户评价与售后类	投诉处理初筛	判断是否需要退款/补偿
	退款申请响应	常规情况自动同意/拒绝
活动与营销类	优惠券创建/停止	满减券、新客券的快速配置
	营销活动报名	识别适合的活动并一键参与
	满减门槛调整	根据客单价动态优化

...		
-----	--	--

4.1 备选1: 营销活动报名

by @王岗(芒鞋)

分块	实现
业务流程摸索及实现 轨迹设计	- 活动感知：遍历活动列表，获取活动和店铺信息匹配情况 - 资格决策：检查商家资质、历史违规记录、活动互斥规则（例如“满减”和“折扣”活动不能同时报名） - 最优化配置：在成本控制前提下最大化曝光 - 配置验证：确认活动生效时间、优惠力度、库存限制，同步商家端展示
环境仿真实现	- 活动列表：返回活动类型、报名条件、预估流量登记、覆盖城市 - 冲突检测器：模拟活动叠加时的风险（如优惠力度过高导致亏损的预测） - 流量模拟器：根据报名活动类型反馈预估曝光
边际场景盘点、测试 样本合成	- 资格瞬变：报名过程中活动名额已满活资质要求变更（如评分骤降） - 时间冲突：活动有效期重叠但优惠互斥，需自动调整排期 - 成本溢出：多活动叠加后优惠力度超过商家预设预算阈值 - 新手期过度：模拟90天扶持期结束前后的策略切换（流量活动 → 常规活动）
任务成功评估标准	

Reference

站点

- [1] [2025 Was Agents. 2026 Is Agent Harnesses. Here's Why That Changes Everything.](#)
- [2] [What Is an Agent Harness? The Infrastructure That Makes AI Agents Actually Work](#)
- [3] [Effective harnesses for long-running agents](#)

[4] <https://github.com/anthropics/claude-quickstarts/tree/main/autonomous-coding>

[5] [Mitchell Hashimoto](#) , [My AI Adoption Journey](#)

[6]  算法——基于claude code框架的业务场景开发 - 原文：场景锚定

[7]  [Harness engineering for coding agent users](#)

论文

[1] Yuxuan Zhang, Haoyang Yu, et. el.. [General Modular Harness for LLM Agents in Multi-Turn Gaming Environments.](#)

Appendix

1. Harnesses有效设计标准参考表

序号	标准	说明
1	Human-in-the-loop controls	Agents在关键决策时暂停（例如删除database、充值、发送邮件等）。Harness需要人工审核。
2	Filesystem access management	Harnesses定义了可访问的目录、被允许的操作，以及冲突解决机制。禁止任务接触到系统文件。
3	Tool call orchestration	糟糕的工具编排会导致无限循环和级联故障。
4	Sub-agent coordination	复杂任务需要特定agents。Harnesses需要协同并管理不同agents之间的交流。
5	Prompt preset management	不同的任务需要不同的指令。例如，代码审查 v.s. 代码生成；bug修复 v.s. 功能开发。Harness维护了prompt库，按需提取。
6	Lifecycle hooks	初始化上下文、运行任务、保存状态、处理故障、重试逻辑、日志记录.....如此种种，Harnesses实现了可靠的workflows。